ASSIGNMENT – 11.5

HT.NO: 2303A52496

BATCH: 50

TASK – 01:

Question:

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class

Stack:

pass

Expected Output:

• A functional stack implementation with all required methods and docstrings.

Prompt: generate a docstring to generate a Stack class with push, pop, peek, and

is_empty methods.

Sample Input Code: class

Stack:

pass

Generated Code:

class Stack:

"""

A Stack class that implements the basic stack operations: push, pop, peek, and is_empty.

Methods:

- push(item): Adds an item to the top of the stack.
- pop(): Removes and returns the item at the top of the stack. Raises an exception if the stack is empty.
- peek(): Returns the item at the top of the stack without removing it. Raises an exception if the stack is empty.
- is_empty(): Returns True if the stack is empty, False otherwise.

```
    """     def
__init__(self):
self.stack = []

    def push(self, item):
        """Adds an item to the top of the stack."""
self.stack.append(item)

    def pop(self):
        """Removes and returns the item at the top of the stack. Raises an exception if the stack
is empty."""         if self.is_empty():
            raise IndexError("pop from empty stack")
return self.stack.pop()

    def peek(self):
        """Returns the item at the top of the stack without removing it. Raises an exception if
the stack is empty."""         if self.is_empty():             raise IndexError("peek from empty
stack")         return self.stack[-1]

    def is_empty(self):
        """Returns True if the stack is empty, False otherwise."""
return len(self.stack) == 0
# Example usage: stack
= Stack() stack.push(1)
stack.push(2)
print(stack.peek())  #
Output: 2
print(stack.pop())   #
Output: 2
```

print(stack.is_empty())

Output: False

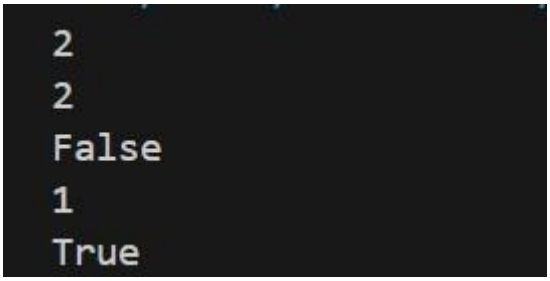
print(stack.pop()) #

Output: 1

print(stack.is_empty())

Output: True

Output:



Explanation:

This code implements a Stack data structure in Python using a list. It provides basic operations like push (add item), pop (remove top item), peek (view top item), and is_empty (check if stack is empty). The example shows how elements are added and removed following LIFO (Last In, First Out) order.

TASK – 02:

Question:

Task: Use AI to implement a Queue using Python lists.

Sample Input Code: class

Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt:

Write a Python program to implement a Queue using lists.

Start with this code: class

Queue:

pass

Implement FIFO operations:

enqueue()

dequeue()

peek() size()

Generated Code: class

Queue:

"""

A Queue class that implements the basic queue operations: enqueue, dequeue, peek, and size.

Methods:

- enqueue(item): Adds an item to the end of the queue.
- dequeue(): Removes and returns the item at the front of the queue. Raises an exception if the queue is empty.
- peek(): Returns the item at the front of the queue without removing it. Raises an exception if the queue is empty.
- size(): Returns the number of items in the queue.

""" def

__init__(self):

self.queue = []

def enqueue(self, item):

"""Adds an item to the end of the queue."""

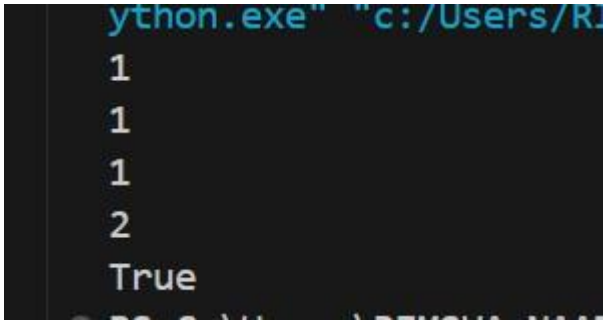
self.queue.append(item)

def dequeue(self):

"""Removes and returns the item at the front of the queue. Raises an exception if the queue is empty.""" if self.is_empty():

```
            raise IndexError("dequeue from empty queue")
return self.queue.pop(0)

    def peek(self):
        """Returns the item at the front of the queue without removing it. Raises an exception if the queue is empty."""         if self.is_empty():             raise IndexError("peek from empty queue")         return self.queue[0]

    def size(self):
        """Returns the number of items in the queue."""
return len(self.queue)

    def is_empty(self):
        """Returns True if the queue is empty, False otherwise."""
return len(self.queue) == 0
# Example usage: queue = Queue()
queue.enqueue(1) queue.enqueue(2)
print(queue.peek())  # Output: 1
print(queue.dequeue())  # Output: 1
print(queue.size())  # Output: 1
print(queue.dequeue())  # Output: 2
print(queue.is_empty())  # Output: True
```

Output:



Explanation:

This Python code creates a Queue class that follows the FIFO (First In, First Out) principle. It provides methods to add items (enqueue), remove items (dequeue), view the first item (peek), and check the size or if the queue is empty.
The example at the end shows how items are added to the queue and removed in the same order.

TASK – 03:

Question:

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code: class

Node:

pass class

LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

Prompt:

generate a Singly Linked List with insert and display methods.

Sample Input Code: class

Node:

pass class

LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method

documentation. generate a python program to implement a Singly Linked List with insert and display methods. **Generated Code:** class Node:

"""

A Node class that represents an individual node in a singly linked list.

```
    Attributes:
        - data: The value stored in the node.
        - next: A reference to the next node in the list (default is None).
    """     def __init__(self,
data):
        self.data = data
self.next = None class
LinkedList:
    """
    A LinkedList class that implements a singly linked list with insert and display methods.

    Methods:
        - insert(data): Inserts a new node with the given data at the end of the list.
        - display(): Prints the elements of the linked list in order.
    """     def
__init__(self):
self.head = None def
insert(self, data):
        """Inserts a new node with the given data at the end of the list."""
    new_node = Node(data)
        if self.head is None:
self.head = new_node
            return
last_node = self.head
while last_node.next:
            last_node = last_node.next
last_node.next = new_node

    def display(self):
```

```
    """Prints the elements of the linked list in order."""
current_node = self.head        while current_node:
        print(current_node.data, end=' ')
current_node = current_node.next        print()  #
for a new line after displaying the list
# Example usage: linked_list =
LinkedList() linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()  # Output: 1 2 3
```

Output:

Explanation:

This code creates a singly linked list using two classes: Node and LinkedList.
The Node class stores data and a reference to the next node, while the LinkedList class allows you to insert new nodes at the end and display all elements in order. The example inserts 1, 2, 3 into the list and prints them as 1 2 3.

TASK – 04:

Question:

Task: Use AI to implement a hash table with basic insert, search, and delete methods. Sample Input Code: class HashTable:

```
    pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Prompt:

Write a Python program to implement a Hash Table with the operations insert, search, and delete.

Use collision handling with chaining (linked lists).

Create a class named HashTable.

Each method should be well commented and easy to understand.

Include a small example showing how to insert, search, and delete elements.

Generated Code: class

Node:

"""

A Node class that represents an individual node in a linked list used for chaining in the hash table.

Attributes:

- key: The key associated with the value stored in the node.

- value: The value stored in the node.

- next: A reference to the next node in the chain (default is None). """

```
def __init__(self, key, value):
    self.key = key
self.value = value
self.next = None class
```

HashTable:

"""

A HashTable class that implements a hash table with chaining for collision handling.

Methods:

- insert(key, value): Inserts a key-value pair into the hash table. If the key already exists, updates its value.

- search(key): Returns the value associated with the given key. Raises an exception if the key is not found.

- delete(key): Removes the key-value pair associated with the given key from the hash table. Raises an exception if the key is not found.

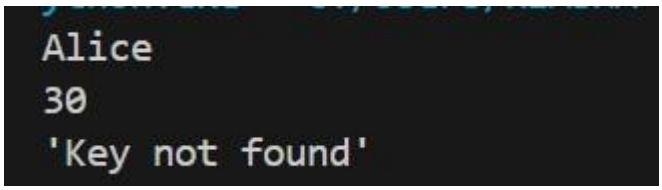
""" def __init__(self,

size=10):

```
        self.size = size         self.table
= [None] * self.size


    def _hash(self, key):
        """Generates a hash for the given key."""
return hash(key) % self.size


    def insert(self, key, value):
        """Inserts a key-value pair into the hash table. If the key already exists, updates its
value."""         index = self._hash(key)         new_node = Node(key, value)         if
self.table[index] is None:
            self.table[index] = new_node
        else:
            current_node = self.table[index]
    while current_node:
                if current_node.key == key:
                    current_node.value = value  # Update existing key
                    return                 if
current_node.next is None:
                    break
                current_node = current_node.next
current_node.next = new_node  # Add new node to the chain


    def search(self, key):
        """Returns the value associated with the given key. Raises an exception if the key is not
found."""         index = self._hash(key)         current_node = self.table[index]         while
current_node:             if current_node.key == key:                 return current_node.value
current_node = current_node.next         raise KeyError("Key not found")
```

```
    def delete(self, key):
        """Removes the key-value pair associated with the given key from the hash table. Raises an exception if the key is not found."""
        index = self._hash(key)
        current_node = self.table[index]
        prev_node = None
        while current_node:
            if current_node.key == key:
                if prev_node:
                    prev_node.next = current_node.next  # Bypass the node to delete it
                else:
                    self.table[index] = current_node.next  # Deleting head of chain
                return prev_node
            current_node = current_node.next
        raise KeyError("Key not found")
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
try:
    print(hash_table.search("name"))  # This will raise an exception
except KeyError as e:
    print(e)  # Output: Key not found
```

Output:



Explanation:

This code implements a Hash Table using chaining to store key-value pairs.
The Node class stores a key, value, and link to the next node, while the HashTable class allows you to insert, search, and delete data using a hash function.
If two keys map to the same index, they are stored in a linked list (chain) at that position.

TASK – 05:

Question:

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Prompt:

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display

connections. generate a python program to implement a graph using an adjacency list with methods to add vertices, add edges, and display connections.

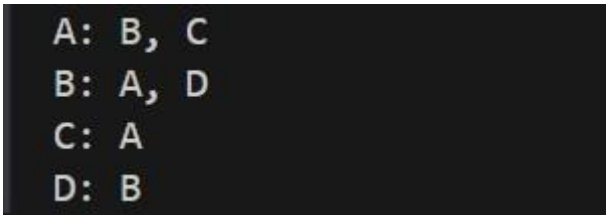
Generated Code: class

Graph:

```
    """
    A Graph class that implements a graph using an adjacency list.


    Methods:
        - add_vertex(vertex): Adds a vertex to the graph.
        - add_edge(vertex1, vertex2): Adds an edge between two vertices in the graph.
        - display(): Displays the connections in the graph.
    """     def
__init__(self):
self.graph = {}
```

```
  def add_vertex(self, vertex):
      """Adds a vertex to the graph."""
if vertex not in self.graph:
self.graph[vertex] = []

   def add_edge(self, vertex1, vertex2):
      """Adds an edge between two vertices in the graph."""
if vertex1 not in self.graph:            self.add_vertex(vertex1)
if vertex2 not in self.graph:            self.add_vertex(vertex2)
self.graph[vertex1].append(vertex2)
self.graph[vertex2].append(vertex1)  # For undirected graph

   def display(self):
      """Displays the connections in the graph."""
for vertex, edges in self.graph.items():
print(f"{vertex}: {', '.join(edges)}")
# Example usage: graph =
Graph()
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "D")
graph.display()
# Output:
# A: B, C
# B: A, D
# C: A
# D: B
```

Output:



Explanation:

This code implements a Graph using an adjacency list in Python.
The class allows you to add vertices, create edges between vertices, and display the connections between them.
In the example, vertices A, B, C, and D are connected, and the display() method prints their relationships.

TASK – 06:

Question:

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time. 5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack o

Queue o

Priority

Queue o

Linked

List o

Binary

Search

Tree

(BST) o

Graph o

Hash

Table o

Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

generate python program for A hospital wants to build a Smart Hospital Management System with the following features: 1. Patient Check-In System

2. Emergency Case Handling

3. Medical Records Storage

4. Doctor Appointment Scheduling

5. Hospital Room Navigation

Choose the most suitable data structure for each feature from this list:

Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

Tasks:

1. Create a table showing: Feature → Chosen Data Structure → Justification.

2. Write 2–3 simple sentences explaining why the data structure is suitable.

3. Implement one feature as a Python program with clear comments and docstrings.

Keep the explanation and code simple and beginner-friendly.

Generated Code:

```
# Task 1: Table of Features and Data Structures feature_data_structure_table

= [

    ("Patient Check-In System", "Queue", "Patients arrive in a sequential manner, and the system needs to process them in the order of arrival."),

    ("Emergency Case Handling", "Priority Queue", "Emergency cases need to be handled based on their severity, with higher-priority cases being treated first."),

    ("Medical Records Storage", "Hash Table", "Medical records need to be accessed quickly using unique patient IDs as keys."),

    ("Doctor Appointment Scheduling", "Linked List", "Appointments need to be managed dynamically, allowing for easy insertion and deletion of scheduling information."),

    ("Hospital Room Navigation", "Graph", "The hospital layout can be represented as a graph where rooms are nodes and corridors are edges.")

]

# Task 2: Explanation of Data Structure Suitability explanations

= [

    "A Queue is suitable for the Patient Check-In System because it follows the First-In-First-Out (FIFO) principle, ensuring that patients are processed in the order they arrive.",

    "A Priority Queue is suitable for Emergency Case Handling because it allows cases to be processed based on their priority level, ensuring that more critical cases are addressed first.",

    "A Hash Table is suitable for Medical Records Storage because it provides fast lookup times when accessing records using unique patient IDs as keys.",

    "A Linked List is suitable for Doctor Appointment Scheduling because it allows for dynamic management of appointments, making it easy to insert and delete scheduling information as needed.",

    "A Graph is suitable for Hospital Room Navigation because it can represent the hospital layout with rooms as nodes and corridors as edges, enabling efficient pathfinding algorithms."

]

# Task 3: Implementing the Patient Check-In System using a Queue class

PatientCheckInSystem:

    """

    A Patient Check-In System that uses a Queue to manage patient arrivals.
```

Methods:

- check_in(patient_name): Adds a patient to the check-in queue.
- process_next_patient(): Processes the next patient in the queue and returns their name. Raises an exception if the queue is empty.
- peek_next_patient(): Returns the name of the next patient to be processed without removing them from the queue. Raises an exception if the queue is empty.
- is_empty(): Returns True if there are no patients in the queue, False otherwise.

""" def __init__(self):
self.queue = []

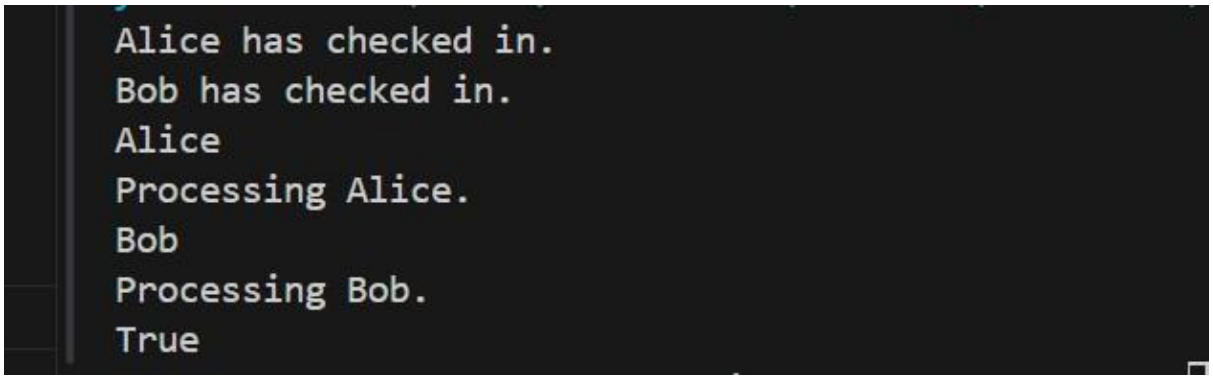
def check_in(self, patient_name):
"""Adds a patient to the check-in queue."""
self.queue.append(patient_name) print(f"{patient_name} has checked in.")

def process_next_patient(self):
"""Processes the next patient in the queue and returns their name. Raises an exception if the queue is empty.""" if self.is_empty():
raise IndexError("No patients to process.")
next_patient = self.queue.pop(0)
print(f"Processing {next_patient}.") return next_patient

def peek_next_patient(self):
"""Returns the name of the next patient to be processed without removing them from the queue. Raises an exception if the queue is empty.""" if self.is_empty():
raise IndexError("No patients to peek.")
return self.queue[0]

```
    def is_empty(self):
        """Returns True if there are no patients in the queue, False otherwise."""
return len(self.queue) == 0
# Example usage: check_in_system = PatientCheckInSystem()
check_in_system.check_in("Alice")
check_in_system.check_in("Bob")
print(check_in_system.peek_next_patient())  # Output: Alice
check_in_system.process_next_patient()  # Output: Processing Alice.
print(check_in_system.peek_next_patient())  # Output: Bob
check_in_system.process_next_patient()  # Output: Processing Bob.
print(check_in_system.is_empty())  # Output: True
```

Output:



Explanation:

Task 1: This code creates a table (list of tuples) that matches hospital system features with the most suitable data structures such as Queue, Priority Queue, Hash Table, Linked List, and Graph.

Task 2: It provides simple explanations of why each data structure is appropriate for those hospital features.

Task 3: It implements a Patient Check-In System using a Queue, where patients are added to the queue when they arrive and processed in FIFO (First-In-First-Out) order.

TASK – 07:

Question:

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details. 4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

A city wants to build a Smart Traffic Management System with these features:

Traffic Signal Queue

Emergency Vehicle Priority Handling

Vehicle Registration Lookup

Road Network Mapping

Parking Slot Availability

Choose the most suitable data structure for each feature from this list:
Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

Tasks:

Create a table showing Feature → Data Structure → Justification.

Write 2–3 simple sentences explaining why the data structure is suitable.

Implement one feature as a Python program with comments and docstrings.

Keep the explanation and code very simple and easy to understand.

Generated Code:

```
# Task 1: Table of Features and Data Structures feature_data_structure_table

= [

    ("Traffic Signal Queue", "Queue", "Traffic signals need to manage vehicles in a sequential manner, allowing them to pass through the intersection in the order they arrive."),

    ("Emergency Vehicle Priority Handling", "Priority Queue", "Emergency vehicles need to be given priority over regular traffic, allowing them to pass through intersections more quickly."),

    ("Vehicle Registration Lookup", "Hash Table", "Vehicle registration information needs to be accessed quickly using unique license plate numbers as keys."),

    ("Road Network Mapping", "Graph", "The road network can be represented as a graph where intersections are nodes and roads are edges, enabling efficient route planning."),

    ("Parking Slot Availability", "Linked List", "Parking slots can be managed dynamically, allowing for easy insertion and deletion of available slots as vehicles park and leave.")

]

# Task 2: Explanation of Data Structure Suitability explanations

= [

    "A Queue is suitable for the Traffic Signal Queue because it follows the First-In-First-Out (FIFO) principle, ensuring that vehicles are processed in the order they arrive at the traffic signal.",

    "A Priority Queue is suitable for Emergency Vehicle Priority Handling because it allows emergency vehicles to be processed based on their priority level, ensuring they can pass through intersections more quickly than regular traffic.",

    "A Hash Table is suitable for Vehicle Registration Lookup because it provides fast lookup times when accessing registration information using unique license plate numbers as keys.",

    "A Graph is suitable for Road Network Mapping because it can represent the road network with intersections as nodes and roads as edges, enabling efficient route planning algorithms.",

    "A Linked List is suitable for Parking Slot Availability because it allows for dynamic management of parking slots, making it easy to insert and delete available slots as vehicles park and leave."

]
```

```
# Task 3: Implementing the Traffic Signal Queue using a Queue class

TrafficSignalQueue:
    """
    A Traffic Signal Queue that uses a Queue to manage vehicles waiting at a traffic signal.

    Methods:
- arrive(vehicle_name): Adds a vehicle to the traffic signal queue.
- process_next_vehicle(): Processes the next vehicle in the queue and returns its name. Raises
  an exception if the queue is empty.
- peek_next_vehicle(): Returns the name of the next vehicle to be processed without
  removing it from the queue. Raises an exception if the queue is empty.
- is_empty(): Returns True if there are no vehicles in the queue, False otherwise.
    """     def
__init__(self):
self.queue = []

    def arrive(self, vehicle_name):
        """Adds a vehicle to the traffic signal queue."""
        self.queue.append(vehicle_name)
print(f"{vehicle_name} has arrived at the traffic signal.")

    def process_next_vehicle(self):
        """Processes the next vehicle in the queue and returns its name. Raises an exception if
the queue is empty."""         if self.is_empty():
            raise IndexError("No vehicles to process.")
next_vehicle = self.queue.pop(0)
print(f"Processing {next_vehicle}.")         return
next_vehicle

    def peek_next_vehicle(self):
```

"""Returns the name of the next vehicle to be processed without removing it from the queue. Raises an exception if the queue is empty.""" if self.is_empty(): raise IndexError("No vehicles to peek.") return self.queue[0]

def is_empty(self):

"""Returns True if there are no vehicles in the queue, False otherwise."""

return len(self.queue) == 0

Example usage: traffic_signal_queue = TrafficSignalQueue()

traffic_signal_queue.arrive("Car A") traffic_signal_queue.arrive("Car B")

print(traffic_signal_queue.peek_next_vehicle()) # Output: Car A

traffic_signal_queue.process_next_vehicle() # Output: Processing Car A.

print(traffic_signal_queue.peek_next_vehicle()) # Output: Car B

traffic_signal_queue.process_next_vehicle() # Output: Processing Car B.

print(traffic_signal_queue.is_empty()) # Output: True **Output:**

```
Car A has arrived at the traffic signal.
Car B has arrived at the traffic signal.
Car A
Processing Car A.
Car B
Processing Car B.
True
```

Explanation:

Task 1: The code creates a table (list of tuples) that links traffic management features with suitable data structures like Queue, Priority Queue, Hash Table, Graph, and Linked List.

Task 2: It explains why each data structure is appropriate for those traffic system features.

Task 3: It implements a Traffic Signal Queue using a Queue, where vehicles join the queue when they arrive and are processed in FIFO (First-In-First-Out) order at the signal.

TASK – 08:

Question:

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID. 5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

An e-commerce company wants to build a Smart Online Shopping System with these features:

Shopping Cart Management

Order Processing System

Top-Selling Products Tracker

Product Search Engine

Delivery Route Planning

Choose the best data structure for each feature from this list:

Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

Tasks:

Create a table showing Feature → Data Structure → Justification.

Write 2–3 simple sentences explaining the choice.

Implement one feature as a Python program with comments and docstrings.

Keep the explanation and code simple and beginner-friendly. generate a python program to implement the Shopping Cart Management feature using a Stack data structure."""

Generated Code:

```
# Task 1: Table of Features and Data Structures feature_data_structure_table

= [

    ("Shopping Cart Management", "Stack", "A stack allows customers to add items to their cart in a last-in-first-out (LIFO) manner, making it easy to manage the most recently added items."),

    ("Order Processing System", "Queue", "Orders need to be processed in the order they were received, making a queue the ideal data structure for this feature."),

    ("Top-Selling Products Tracker", "Hash Table", "A hash table allows for fast access and updates of product sales data using product IDs as keys."),

    ("Product Search Engine", "Binary Search Tree (BST)", "A BST allows for efficient searching of products based on attributes like price or name."),

    ("Delivery Route Planning", "Graph", "The delivery routes can be represented as a graph where locations are nodes and routes are edges, enabling efficient route planning algorithms.")

]

# Task 2: Explanation of Data Structure Suitability explanations

= [

    "A Stack is suitable for Shopping Cart Management because it allows customers to manage their cart in a last-in-first-out (LIFO) manner, making it easy to add and remove the most recently added items.",

    "A Queue is suitable for the Order Processing System because it ensures that orders are processed in the order they were received, following the First-In-First-Out (FIFO) principle.",

    "A Hash Table is suitable for the Top-Selling Products Tracker because it provides fast access and updates of product sales data using product IDs as keys.",
```

"A Binary Search Tree (BST) is suitable for the Product Search Engine because it allows for efficient searching of products based on attributes like price or name.",

"A Graph is suitable for Delivery Route Planning because it can represent delivery routes with locations as nodes and routes as edges, enabling efficient route planning algorithms."

]

Task 3: Implementing the Shopping Cart Management using a Stack class

ShoppingCart:

"""

A Shopping Cart class that uses a Stack to manage items in the cart.

Methods:

- add_item(item): Adds an item to the shopping cart.
- remove_last_item(): Removes and returns the last item added to the cart. Raises an exception if the cart is empty.
- peek_last_item(): Returns the last item added to the cart without removing it. Raises an exception if the cart is empty.
- is_empty(): Returns True if there are no items in the cart, False otherwise.

""" def

__init__(self):

self.cart = []

def add_item(self, item):

"""Adds an item to the shopping cart."""

self.cart.append(item) print(f"{item} has been added

to the shopping cart.")

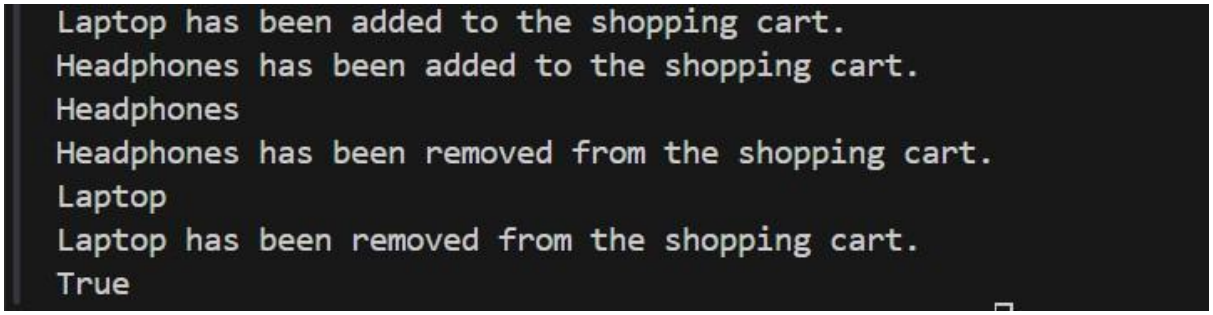
def remove_last_item(self):

"""Removes and returns the last item added to the cart. Raises an exception if the cart is empty."""

if self.is_empty():

```
        raise IndexError("No items to remove.")        last_item =
self.cart.pop()        print(f"{last_item} has been removed from the
shopping cart.")        return last_item


    def peek_last_item(self):
        """Returns the last item added to the cart without removing it. Raises an exception if the
cart is empty."""        if self.is_empty():            raise IndexError("No items to peek.")
return self.cart[-1]


    def is_empty(self):
        """Returns True if there are no items in the cart, False otherwise."""
return len(self.cart) == 0 # Example usage:
shopping_cart = ShoppingCart() shopping_cart.add_item("Laptop")
shopping_cart.add_item("Headphones")
print(shopping_cart.peek_last_item())  # Output: Headphones
shopping_cart.remove_last_item()  # Output: Headphones has been removed from the
shopping cart print(shopping_cart.peek_last_item())  # Output: Laptop
shopping_cart.remove_last_item()  # Output: Laptop has been removed from the shopping
cart print(shopping_cart.is_empty())  # Output: True
```

Output:



Explanation:

Task 1: The code creates a table (list of tuples) that links e-commerce system features with suitable data structures such as Stack, Queue, Hash Table, Binary Search Tree, and Graph.

Task 2: It provides short explanations of why each data structure fits those features.

Task 3: It implements Shopping Cart Management using a Stack, where items are added and removed using the LIFO (Last-In-First-Out) principle, meaning the most recently added item is removed first.